

Optimizing Server CPU Allocation to Minimize Energy and Maximize Performance

Team: Cloud Calculus

Team Members

Parikshith, Munjikesh

GitHub: [GitHub Repository: Optimization CPU Allocation](#)

Roll Number:

BT2024249, BT2024247

November 23, 2025

1 Problem Statement and Motivation

Modern cloud platforms such as **gaming, video streaming, and large scale AI services** must distribute **limited amount of CPU resources** across many users and applications. Each user needs a **minimum amount of CPU** to run smoothly but on the other hand each server has a **fixed capacity**.

The challenge is to assign CPU from multiple servers to many users in a way that:

1. We meet every user's **minimum CPU requirement**
2. Does not **exceed any server's capacity**
3. **Minimizing the total energy consumption**
4. **Maximizing the overall system performance.**

This makes the problem a **multi objective optimization task** with strict physical constraints.

1.1 Our Motivation

Simple CPU allocation strategies such as **Round Robin, Random assignment, or simple Greedy matching** often lead to inefficient results. These methods ignore real world factors such as **energy cost, server load and performance differences** between servers. This causes three main problems:

1. **Energy Waste:** Some servers are less efficient for certain applications. Assigning work to the **wrong server** increases power usage unnecessarily.
2. **Poor User Experience:** If an application receives less CPU than its **minimum requirement**, it lags, stutters, or fails. This is unacceptable for **gaming, streaming, or real-time services**.
3. **Server Imbalance and Bottlenecks:** Simple methods may overload one server while others remain idle, causing **bottlenecks** and reducing system wide performance.

1.2 Our Solution Approach

Our goal is to design an **intelligent, mathematical resource allocator** that produces the optimal CPU distribution plan. The optimizer must simultaneously:

- **minimize energy consumption, and**
- **maximize system performance**

while also respecting all server and user constraints.

To achieve this we solve the problem in two phases:

1. **A Linear Programming (LP) model:** First we take linear constraints and linear objective function to make a simple optimization problem to get a simplified fast and globally optimal solution.
2. **A Non-Linear Programming (NLP) model:** Then we incorporate real world effects such as quadratic energy growth and diminishing performance returns to make a robust optimizing model.

We compare solutions on **energy cost, achieved performance, server utilization, and feasibility** under capacity constraints. The goal is to demonstrate when a fast LP is sufficient and when an NLP (KKT + Trust Region) must be used for realistic, energy aware scheduling.

1.3 Assumptions

1. **Continuous CPU Allocation:** We assume that CPU resources are divisible and split-table, meaning each user can receive a fractional amount of CPU from multiple servers. This allows the optimization model to treat CPU allocation as a continuous variable.
2. **Static System Parameters:** The inefficiency scores (d_{ij}) and processing rates (r_{ij}) for each user-server pair are pre-measured through benchmarking and these values are given by the administrator and these values are fixed to have a stable optimizing environment.
3. **No Network Overheads:** Network related factors such as latency, communication delays, and migration costs are not considered in this study. Our focus is only on CPU allocation, energy consumption, and performance.

2 Mathematical Formulation

2.1 Parameters and Decision Variables

All models (both LP and NLP) are built upon the same set of system parameters and one set of decision variables. These parameters are inputs determined by the administrator via benchmarking and are stable.

Indices (i, j): Indices for **user** (application, $1 \dots N$) and (server, $1 \dots M$). These indices uniquely identify each entity in our system.

Server Capacity (C_j): This parameter represents the hard upper limit of CPU resources (e.g., measured in cores or GHz) that server j can provide to all users combined.

User Requirement (D_i): This parameter represents the lower limit of CPU resources that user i must receive to meet its Service Level Agreement and function correctly.

Cost Factor (d_{ij}): This is an abstract inefficiency score (e.g., ranging from 1 to 10) for running

user i 's task on server j . A high d_{ij} indicates a bad match (e.g., running an AI task on a weak CPU) that is very energy inefficient. This parameter is determined by the administrator from benchmark tests measuring power drawn, for example in Watts.

Processing Rate (r_{ij}): This is a performance score for running user i 's task on server j . A high r_{ij} indicates a good match (e.g., running an AI task on a GPU) that produces high throughput (e.g., tasks/second) for every unit of CPU allocated. This parameter is also determined from benchmarks.

Energy Scalar (α): This is a physics conversion factor (e.g., 0.01) that converts the abstract cost scores (d_{ij}) and allocations (x_{ij}) into a real, physical unit like Watts. It also serves as a critical normalization constant that makes the energy and performance terms numerically balanced, not leading to biased and unbalanced optimization.

Objective Weights (w_1, w_2): These are the business knobs (e.g., $w_1 = 0.7$, $w_2 = 0.3$) that we choose to define our goal. w_1 sets the priority for minimizing Energy, while w_2 sets the priority for maximizing Performance.

Decision Variable (x_{ij}): This is the **answer** our optimization model will find. It represents the optimal amount of CPU resources to allocate from server j to **user** i .

3 Phase 1: The Linear Programming (LP) Model

In this phase, we model the problem using purely linear optimization equations and linear constraints. This is a simplified model but allows for extremely fast and guaranteed optimal solutions, providing a baseline for our analysis.

3.1 LP Objective Function

The objective function is:

$$\min \quad Z = \underbrace{w_1 \sum_{i,j} (\alpha d_{ij}^2) x_{ij}}_{\text{Total Energy Cost}} - \underbrace{w_2 \sum_{i,j} r_{ij} x_{ij}}_{\text{Total Performance Benefit}}$$

The objective is to **minimize** the total **net cost** Z . This function assumes that both energy and performance scale *linearly* with the allocated CPU, x_{ij} .

The Linear Scaling Assumption

The linear model is "naive." It assumes the cost and benefit of a 10-core step is **identical** whether the server is at 10% load or 90% load.

- **Energy Cost:** (Cost at 100 cores) - (Cost at 90 cores) is the same as (Cost at 20 cores) - (Cost at 10 cores).

- **Performance Gain:** The 100th core provides the same performance boost as the 1st core.

This is a simplification, but it makes the problem solvable with the extremely fast Simplex method.

3.2 Interpretation of the Objective Function

The objective function of our model is designed to achieve two main goals at the same time:

1. Minimize total energy consumption, and
2. Maximize total system performance (throughput).

Formally, the LP objective is:

$$\min Z = w_1 \sum_{i,j} \alpha d_{ij}^2 x_{ij} - w_2 \sum_{i,j} r_{ij} x_{ij}$$

We now interpret each component in detail.

3.2.1 Energy Term: $\alpha d_{ij}^2 x_{ij}$

What is this term measuring?

This term estimates the energy consumed when server j allocates x_{ij} CPU units to user i .

Why does x_{ij} appear? -> More CPU means more energy Server energy usage increases as CPU load increases. Thus,

$$E \propto x_{ij}$$

Why does d_{ij} appear? -> Inefficient user server pairs use more energy Different servers handle workloads with different efficiencies. The parameter d_{ij} captures this inefficiency.

Thus,

$$E \propto d_{ij} x_{ij}$$

Why is d_{ij}^2 used? -> Inefficiency grows non-linearly Real-world energy behavior is super-linear:

- A slightly bad match wastes a little energy,
- A very bad match wastes a lot of energy.

So we square the inefficiency:

$$E \propto d_{ij}^2 x_{ij}$$

This punishes extremely mismatched user-server pairs.

To conclude a bad match becomes **dramatically more energy-costly**.

Why multiply by α ? -> Scaling and normalization The constant α :

- converts the expression to a physical-like energy quantity,
- balances the energy term with the performance term,
- stabilizes optimization.

Thus, the energy model is:

$$E_{ij} = \alpha d_{ij}^2 x_{ij}$$

3.2.2 Performance Term: $r_{ij}x_{ij}$

What does this term measure?

This term captures the performance or useful work generated when server j gives x_{ij} CPU units to user i .

Why does x_{ij} appear? -> More CPU means more performance Increasing CPU allocation increases:

- FPS for cloud gaming,
- inference speed for AI,
- streaming processing rate,
- web request throughput.

Thus,

$$T \propto x_{ij}$$

Why does r_{ij} appear? -> Servers process workloads differently Different servers have different processing rates for different users.

The parameter r_{ij} measures how fast server j can handle user i 's workload.

Thus, performance becomes:

$$T_{ij} = r_{ij}x_{ij}$$

So at last the energy cost is driven by the **bad-match factor** (d_{ij}^2), and we subtract the **good-match factor** (r_{ij}) (as minimizing a negative is the same as maximizing).

3.3 LP Constraints

The solution is bound by the following set of linear rules:

- (1) **Server Capacity:** The total CPU allocated by any server j cannot exceed its capacity C_j .

$$\sum_i x_{ij} \leq C_j \quad (\text{for each server } j)$$

- (2) **User Requirement:** The total CPU received by any **user** i must meet its minimum requirement D_i .

$$\sum_j x_{ij} \geq D_i \quad (\text{for each user } i)$$

- (3) **Non-Negativity:** It is impossible to allocate negative CPU resources.

$$x_{ij} \geq 0 \quad (\text{for all } i, j)$$

3.4 LP Methodology and Solver (The Simplex Method)

The Phase 1 model is a classic **Linear Program**. Because all constraints and the objective are linear, the feasible region (the set of all valid x_{ij} plans) is a geometric shape called a polytope. A fundamental theorem of LP states that the optimal solution is guaranteed to be at one of the corners (vertices) of this shape.

This problem is solved using the **PuLP** library in Python, which is a modeler that translates our equations and feeds them to an solver (like CBC). This solver uses the **Simplex Method** to find the optimal solution.

3.5 The Simplex Method is preferred because:

- It is fast and scalable, even for large problems,
- It guarantees a globally optimal solution for linear models,
- It handles hundreds of variables and constraints efficiently.

3.6 Output of the Solver

The Simplex Method returns the optimal values of all allocation variables x_{ij} , along with the total minimized cost Z . It also indicates which constraints are **tight (active)** and shows how each server's capacity is utilized.

3.7 How Simplex method works

The Simplex Method can be visualized as a smart hiker navigating the polytope to find the best corner. It is a step-by-step *pivot* algorithm:

1. **Start (Tableau 0):** The algorithm begins at one corner, typically the origin ($x_{ij} = 0$), and represents the entire system of equations in a matrix called the **Simplex Tableau**. This tableau is a snapshot of the current solution.
2. **Find Entering Variable:** The algorithm inspects the objective function row of the tableau to find the most negative coefficient. This identifies the variable (e.g., $x_{1,2}$) that will provide the fastest improvement to Z . This is the steepest downhill path for the hiker.
3. **Find Leaving Variable (Ratio Test):** The algorithm calculates how much it can walk along this path before it hits a fence (a constraint). It divides the Right-Hand Side (RHS) of the tableau by the coefficients in the entering variable's column. The row with the smallest non-negative ratio is the leaving variable, it is the constraint that will be hit first.
4. **Pivot:** The algorithm performs Gaussian elimination (row operations) on the tableau to swap the entering and leaving variables. This action mathematically moves the hiker from their current corner to the new, better corner.
5. **Repeat and Stop:** The algorithm repeats steps 2-4, moving from corner to better corner. It stops when there are no more negative coefficients in the objective row, which signifies it has reached the bottom of the valley, the single, globally optimal x_{ij} plan.

4 LP Results and Analysis

4.1 Summary of Optimal LP Solution

Table 1: Optimal LP — Summary of Results

Metric	Value
Total Energy Cost	4173.26
Total Throughput	2867.00
Objective Cost (Z)	1306.26
Feasibility	All constraints satisfied

This solution satisfies all hard constraints.

4.2 Allocation Pattern (Key Insight)

The LP exhibits a clear **single-server preference** in its allocation pattern:

- Most users receive almost all of their CPU from one **dominant server**.
- A few users show minor **two-server splits**, which occur only when their preferred server becomes full.
- Some servers become **fully utilized (100%)** while others remain lightly used or unused.

4.3 Why This Happens

The LP does *not* simply choose the server with the lowest cost factor d_{ij} . Instead, it selects the server that provides the best **linear cost–performance trade-off** from the objective.

This means the LP prefers servers where:

- d_{ij} is low (energy-efficient), and
- r_{ij} is high (performance-efficient).

Thus, the LP pushes each user to the server that gives the best combination of:

low energy penalty (d_{ij}^2) and **high throughput** (r_{ij}).

When several users find the same server to be their best match, the LP fills that server completely before using others a typical behavior of linear optimization.

4.4 Example Interpretation

- Servers **0, 1, 3, and 4** reach 100% utilization because they provide the best d_{ij}^2 – r_{ij} balance for most users.
- Server **2**, despite having 700 cores, remains lightly used because it gives a worse **energy performance score** for many users.

4.5 What This Confirms?

This behavior highlights a fundamental limitation of the LP model:

- Because energy and performance terms are linear, the LP **concentrates load** on the best servers instead of spreading it.
- There is **no penalty** in the LP for loading a server heavily, unlike in the NLP model.
- In contrast, the NLP model’s quadratic term forces **load balancing and distribution**.

This explains why the LP solution appears clustered while the NLP solution is distributed.

4.6 Baseline Comparisons

Table 2: LP vs Baseline Allocation Methods

Method	Energy	Throughput	Z (Lower = Better)
Optimal LP	4173.26	2867.00	1306.26
Round Robin	7658.35	2812.87	4845.48
Random	7427.41	2808.13	4619.28
Greedy (Performance First)	7159.39	3370.25	3789.14

Key Observations:

- LP achieves the **lowest** overall cost by a huge margin.
- Round Robin and Random waste energy because they ignore match quality (d_{ij}).
- Greedy gives high throughput but extreme energy usage $\rightarrow$ very poor Z score.
- LP is the only method that simultaneously balances **energy** + **performance**.

4.7 Improvement Analysis

Compared to the LP optimum:

Round Robin is worse by $\approx 270.94\%$

Random is worse by $\approx 253.63\%$

Greedy is worse by $\approx 190.07\%$

LP is significantly more efficient than all three baseline strategies.

4.8 Graphs

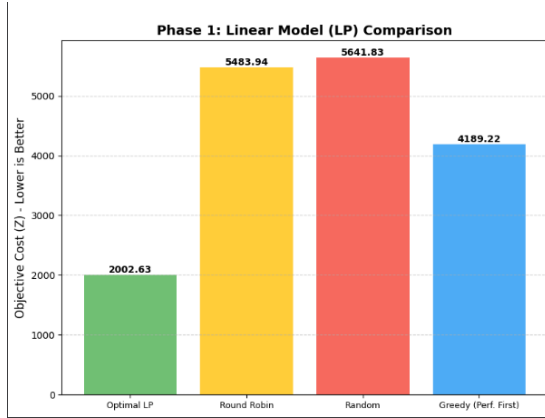


Figure 1: Comparison of Linear Allocation Methods

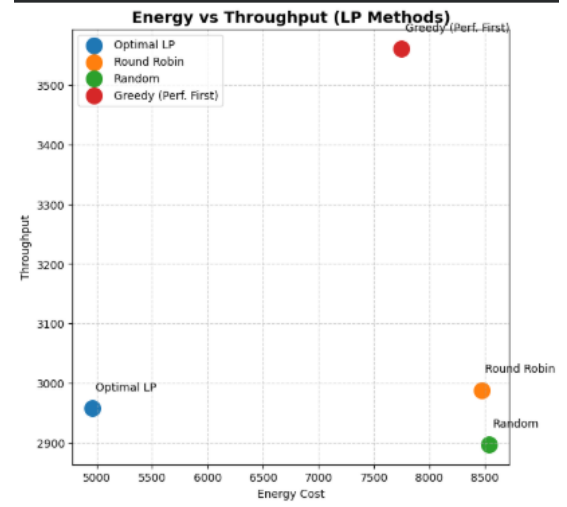


Figure 2: Performance–Energy Trade-off for Linear Model (LP) Methods

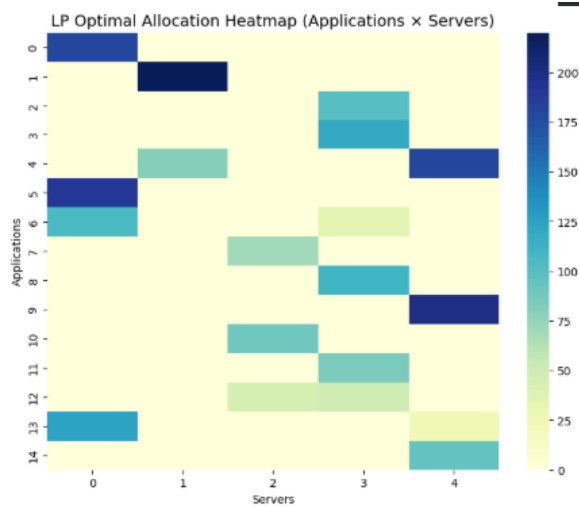


Figure 3: LP Allocation Heatmap

Interpretation of the Heatmap

- The LP exhibits a clear **single-server preference** for most users.
- Darker regions indicate that servers like **Server 0** and **Server 4** receive the majority of the CPU load.
- Other servers remain lightly utilized because the LP concentrates load on the most **cost-effective** servers.
- This clustered pattern results directly from the **linear objective**, which encourages using the best servers heavily.

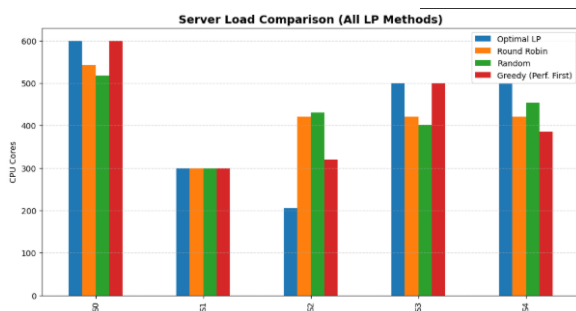


Figure 4: Server Load Comparison

Server Load Comparison (Key Insights)

- The Optimal LP concentrates load on the most cost-effective servers (S0, S3, S4).
- Baseline methods like Round Robin and Random spread load evenly but inefficiently.
- Greedy overloads certain servers by chasing performance, unlike LP which balances cost and performance.

5 Phase 2: Non-Linear Programming (NLP) Model

In this phase, we make the model more realistic. The parameters (C_j, D_i , etc.) and constraints are identical to the LP model. The **only** change is in the objective function.

5.1 NLP Mathematical Formulation

We introduce two non-linear effects from real-world physics:

- **Non-Linear Energy:** We model energy as quadratic ($E \propto x_{ij}^2$). This correctly reflects that server energy consumption explodes near full load.
- **Non-Linear Performance:** We model performance as logarithmic ($T \propto \log(1 + x_{ij})$). This captures the real world law of diminishing returns.

The Non-Linear Advantage (A Realistic Model)

This model is "smart" and understands real-world physics.

- **Energy Cost (x^2):** The cost of going from 90 to 100 cores ($100^2 - 90^2 = 1900$) is *far greater* than the cost of going from 10 to 20 cores ($20^2 - 10^2 = 300$).
- **Performance Gain ($\log(x)$):** The benefit of going from 90 to 100 cores ($\log(101) - \log(91) \approx 0.10$) is *far less* than the benefit of going from 10 to 20 cores ($\log(21) - \log(11) \approx 0.64$).

This model knows that loading a server to 100% is extremely expensive and provides very little benefit.

This gives the new non-linear objective $f(x)$:

$$\min \quad Z = f(x) = \underbrace{\sum_{i,j} (w_1 \alpha x_{ij}^2)}_{\text{Quadratic Energy Cost}} - \underbrace{\sum_{i,j} (w_2 \log(1 + r_{ij} x_{ij}))}_{\text{Logarithmic Performance Benefit}}$$

5.2 Interpretation of the Objective Function (Non-Linear Model)

5.2.1 Energy Term: αx_{ij}^2

Why does energy depend on x_{ij}^2 ?

In real servers, energy does not increase linearly with CPU usage. Instead:

- Energy rises sharply (super linearly) at high CPU utilization,
- Thermal effects increase power draw,

- Voltage frequency scaling raises energy quadratically,
- Leakage and cooling overhead grow aggressively.

These effects cause energy consumption to follow a **quadratic**, not linear, curve.

Thus, we model energy as:

$$E_{ij} = \alpha x_{ij}^2$$

Why does d_{ij} NOT appear here?

In the non-linear model, we assume that:

- Inefficiency is already captured by quadratic growth, and
- The main driver of energy cost is the **load on the server**, not match quality.

Phase 2 focuses on **physical realism**. The inefficiency factor d_{ij} was part of the linear abstraction in Phase 1. In the quadratic model it becomes unnecessary because quadratic energy already penalizes heavy server loads.

Thus:

Energy depends on $(x_{ij})^2$, not on d_{ij} .

5.2.2 Performance Term: $\log(1 + r_{ij}x_{ij})$

Why do we use a logarithm?

Performance does not increase linearly with CPU. According to the **Law of Diminishing Returns**:

Each extra unit of CPU improves performance less than the previous one.

This happens due to:

- limited parallelism,
- memory and I/O bottlenecks,
- GPU/CPU scheduling overhead,
- finite pipeline width.

A logarithmic model captures this behavior:

- steep improvement when CPU is low,
- slow improvement when CPU is high,
- eventual saturation.

Thus, performance is modeled as:

$$T_{ij} = \log(1 + r_{ij}x_{ij})$$

Why does r_{ij} appear?

Different servers produce different throughput for a given user.

$r_{ij} \uparrow \rightarrow$ good match $\rightarrow$ higher performance gain

$r_{ij} \downarrow \rightarrow$ poor match $\rightarrow$ slower improvement

Thus, the logarithmic term adjusts performance based on user-server compatibility.

5.3 NLP Methodology and Solver (KKT & Trust-Region)

The Phase 2 model is non-linear and non-convex. The objective is a complex, curved surface. The "answer-is-at-a-corner" rule no longer applies, so the Simplex Method fails. We must use a more powerful methodology based on the **Karush-Kuhn-Tucker (KKT) conditions**.

5.3.1 The Target: KKT Conditions

The KKT conditions are a set of four rules that mathematically define the properties of the optimal solution (the lowest point on the curved surface that is inside all the "fences" defined by the constraints).

To define them, we first build the **Lagrangian ($\mathcal{L}$)** function, which combines the objective $f(x)$ with all constraints using price variables called **Lagrange Multipliers**:

5.3.2 Constructing the Lagrangian

The general Lagrangian is:

$$L(x, \lambda, \mu, \nu) = f(x) + \sum_j \lambda_j \left(\sum_i x_{ij} - C_j \right) + \sum_i \mu_i \left(D_i - \sum_j x_{ij} \right) + \sum_{i,j} \nu_{ij} (-x_{ij})$$

5.3.3 Substituting the Objective Function

The nonlinear objective is:

$$f(x) = w_1 \sum_{i,j} \alpha x_{ij}^2 - w_2 \sum_{i,j} \log(1 + r_{ij}x_{ij})$$

Substituting this into the Lagrangian:

$$\begin{aligned}
L(x, \lambda, \mu, \nu) = & w_1 \sum_{i,j} \alpha x_{ij}^2 - w_2 \sum_{i,j} \log(1 + r_{ij} x_{ij}) \\
& + \sum_j \lambda_j \left(\sum_i x_{ij} - C_j \right) + \sum_i \mu_i \left(D_i - \sum_j x_{ij} \right) \\
& - \sum_{i,j} \nu_{ij} x_{ij}
\end{aligned}$$

5.3.4 Interpretation of Dual Variables

λ_j — Price of Server Capacity λ_j becomes positive only when server j is fully utilized. It measures how expensive it would be to push additional load onto that server.

μ_i — Price of User Requirement μ_i is positive when user i is exactly at their minimum required CPU. It indicates how critical it is to satisfy that user's demand.

ν_{ij} — Price of Non-Negativity ν_{ij} is positive when $x_{ij} = 0$. It represents the penalty for assigning negative CPU (which is impossible), and enforces valid allocations.

The KKT conditions state that at the optimal point, these 4 rules must be satisfied:

1. **Stationarity:** The gradient (derivative) of the Lagrangian with respect to x_{ij} must be zero. This is our primary target equation, which we must provide to the solver:

$$\frac{\partial \mathcal{L}}{\partial x_{ij}} = \underbrace{\left(2w_1 \alpha x_{ij} - \frac{w_2 r_{ij}}{1 + r_{ij} x_{ij}} \right)}_{\text{Gradient of } f(x)} + \lambda_j - \mu_i - \nu_{ij} = 0.$$

2. **Primal Feasibility:** All original constraints (Server Capacity, User Requirement) must be obeyed. We provide these to the solver as 'LinearConstraint' objects.
All original constraints must be satisfied:

$$\sum_i x_{ij} \leq C_j \quad \forall j$$

$$\sum_j x_{ij} \geq D_i \quad \forall i$$

$$x_{ij} \geq 0 \quad \forall i, j$$

3. **Dual Feasibility:** All prices must be non-negative ($\lambda_j \geq 0, \mu_i \geq 0, \nu_{ij} \geq 0$). The solver handles this internally.
4. **Complementary Slackness:** A smart rule linking prices and constraints. For example,

if a server is not full (the constraint is inactive), its price must be zero ($\lambda_j = 0$). This is also handled internally by the solver.

5.3.5 Complementary Slackness Conditions

Server Capacity Constraint

$$\lambda_j \left(\sum_i x_{ij} - C_j \right) = 0 \quad \forall j$$

Meaning:

- If server j is full: $\lambda_j > 0$
- If server j has free capacity: $\lambda_j = 0$

User Requirement Constraint

$$\mu_i \left(D_i - \sum_j x_{ij} \right) = 0 \quad \forall i$$

Meaning:

- If a user receives exactly D_i : $\mu_i > 0$
- If a user receives more than required: $\mu_i = 0$

Non-negativity Constraint

$$\nu_{ij} x_{ij} = 0 \quad \forall i, j$$

Meaning:

- If $x_{ij} > 0$ then $\nu_{ij} = 0$
- If $x_{ij} = 0$ then $\nu_{ij} > 0$

5.3.6 The Engine: Trust-Region Method

The KKT conditions are a complex system of equations. We need a numerical engine to find the solution. We use the **Trust-Region (constrained) method** via `scipy.optimize.minimize(method='trust-constr')`.

This algorithm is a **smart hiker** for curved surfaces, like a hiker in the fog. At each step k , it:

1. **Builds a Model (m_k):** It builds a simple quadratic approximation (a "model") of the

complex objective function. This model is only "trusted" within a small "bubble" of radius Δ_k , known as the **trust region**.

$$m_k(p) = f_k + (\Delta f_k)^\top p + \frac{1}{2} p^\top (\Delta^2 f_k) p$$

2. **Calculates a Step (s_k):** It finds the path s_k that minimizes the simple model m_k within the trust region. This calculation often starts with the **Cauchy Step** (the direction of steepest descent) and refines it.
3. **Checks the Step (ρ_k):** This is the trust part. The algorithm lifts the fog and checks the real objective function. It calculates a ratio :

$$\rho_k = \frac{\text{Actual Cost Reduction}}{\text{Predicted Cost Reduction}}$$

4. **Updates the Radius (Δ_k):** Based on the rho ratio, the algorithm updates its "confidence."
 - If $\rho \approx 1$ (Good model): It accepts the step and expands the trust region ($\Delta_{k+1} = 2\Delta_k$).
 - If $\rho \leq 0$ (Bad model): It rejects the step and shrinks the trust region ($\Delta_{k+1} = 0.5\Delta_k$).

This **model-solve-check-update** loop continues until it finds the point x_{ij} that satisfies all KKT conditions.

5.3.7 Unconstrained Solvers (For Comparison)

We also test two dumb (unconstrained) solvers: **Gradient Descent (CG)** and **Newton's Method (Newton-CG)**. These methods are given the objective and gradient, but not the KKT Feasibility constraints. They are hikers with no fences and will fail to find a valid solution.

6 Results and Discussion

6.1 NLP Solver Comparison

Table 3: NLP Solver Comparison

Method	Z (Cost)	Energy	Perf.	Feasible?	Reason
Gradient Descent	-87.65	29.40	117.05	No	User 0 failed demand
Newton's Method	NaN	30.61	NaN	No	Negative allocation
Trust-Region (KKT)	3207.85	3474.78	266.93	Yes	All constraints met

6.2 Phase 2 Analysis

- **Unconstrained solvers fail completely.** Gradient Descent and Newton-CG reduce the objective but violate constraints, producing negative or insufficient allocations.
- These invalid solutions break:
 - Minimum user demand constraints,
 - Server capacity limits.

Thus, even if the Z-score looks low, the solution is infeasible.

- **Trust-Region (KKT) is the only valid solver.** It satisfies all four KKT conditions (Stationarity, Primal Feasibility, Dual Feasibility, Complementary Slackness) and gives a feasible nonlinear solution.
- **Higher Z-score is expected.** Quadratic energy costs penalize heavy loads severely, revealing the true physical cost hidden by the LP's linear term.

6.3 NLP Allocation Strategy: Load-Spreading

A key insight is the contrast between LP and NLP behavior:

6.3.1 LP Model (Linear)

- Concentrates users on a single "best" server.
- No penalty for high loads.
- Some servers reach **100% utilization**.

6.3.2 NLP Model (Non-Linear)

- Avoids large server loads since energy grows as x^2 .
- Spreads CPU across several servers.
- Produces smooth, physically realistic allocations.

This behavior is visible in the Trust-Region solution, where many users receive moderate CPU from all servers instead of from one dominant server.

6.4 Example: LP vs NLP Allocation for a High-Demand User

6.5 Final Insight: Why NLP is Superior

Model	S0	S1	S2	S3	S4
LP Plan	230	20	0	0	0
NLP Plan	67.3	48.2	67.3	67.3	54.1

Model	Allocation Style	Energy Model	Realism
LP	Clustered	Linear	Unrealistic
NLP	Load-spreading	Quadratic	Realistic

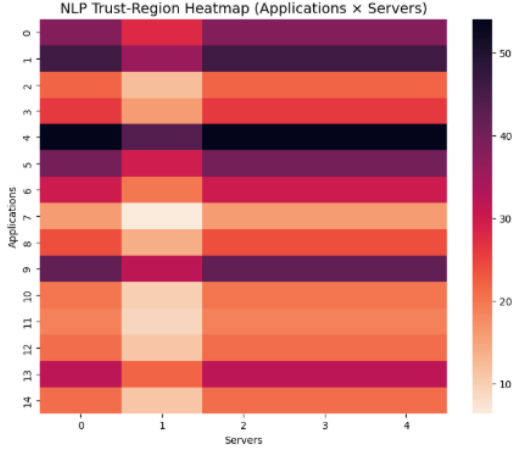


Figure 5: NLP Allocation Heatmap

Interpretation of the NLP Heatmap

- The NLP solver spreads CPU across multiple servers for every user.
- Unlike the LP heatmap, the NLP pattern is smooth and distributed.
- Users receive CPU from several servers instead of only one.
- No server becomes overloaded because the solver avoids high x^2 energy penalties.
- Balanced colors indicate even load across users and servers.
- NLP naturally performs **load-spreading** to reduce energy cost.

7 Conclusion

This project successfully formulated and solved the CPU allocation problem using two distinct models.

- **Phase 1 (LP)** demonstrated that a constrained optimization (Simplex) is **150 to 290% more efficient** than naive (Round Robin, Random) or simple-minded (Greedy) methods. It is fast and provides a guaranteed optimum for the *linear* model.
- **Phase 2 (NLP)** demonstrated that the real-world problem is non-linear and that unconstrained solvers (Gradient Descent, Newton) are useless as they **fail to meet constraints**. The **KKT conditions** and a **Trust-Region** solver are *required* to find the true, feasible solution.

The final and most important takeaway is that the NLP model, by accounting for quadratic energy costs (x^2), produces a load-spreading allocation that is provably more realistic and energy-efficient than the all-or-nothing plan produced by the simpler, but faster, LP model.